

Agentic Scrum

Lo sviluppatore come Product Owner, gli agenti AI come team — un metodo
human-in-the-loop, board-driven, per lo sviluppo software autonomo

Antonio Riccio · DomoticLab

Whitepaper v1 · 8 luglio 2026

Indice

Sintesi	2
1. Il problema	2
2. Il metodo	2
3. La pipeline	2
4. Perché funziona	3
5. Benefici	3
6. Limiti onesti	3
7. Conclusione	4
Licenza e citazione	5

Sintesi

Gli agenti sanno ormai scrivere, testare e rilasciare codice vero. La domanda non è più *se* sappiano programmare, ma *come l'umano resti in controllo* senza diventare il collo di bottiglia. **Agentic Scrum** risponde invertendo i ruoli: l'umano fa da **Product Owner / Scrum Master**, gli agenti AI fanno da **team di sviluppo**. L'interfaccia tra i due è una normale **board Jira**, e il controllo human-in-the-loop è codificato in un segnale unico e visibile — **l'assegnatario della card**. Il risultato è una pipeline completa (ideazione, planning, sviluppo, review, rilascio) che gira in autonomia ma non avanza mai oltre una decisione umana senza un handoff esplicito.

Il nome colloca il metodo dentro termini già affermati nel 2026 — *AI-Augmented Scrum*, *agentic coding / agentic SDLC*, *loop engineering* — mentre il suo contributo distintivo è l'uso dell'**assegnatario della card come testimone del controllo umano**. Implementazione di riferimento: *Spoolio*, open source.

1. Il problema

Due modi di fallire dominano i setup “l'AI scrive il codice”:

1. **L'umano è il collo di bottiglia** — ogni passo richiede un prompt, un copia-incolla, una babysitting. L'agente è veloce; l'umano è il tetto di throughput.
2. **L'umano perde il controllo** — l'agente corre libero e mergia codice discutibile, e nessuno sa *di chi è il turno* o *cosa serva ancora all'umano*.

Agentic Scrum elimina entrambi rendendo il controllo **strutturale** invece che manuale.

2. Il metodo

Ruoli. Lo sviluppatore non scrive codice: fa sprint planning, priorità, review, decide i rilasci — il lavoro di un PO/Scrum Master. Gli agenti fanno ideazione, planning, coding e integrazione.

La board è l'interfaccia. Il lavoro vive su una board Jira con le colonne consuete (Bug, Idea, Todo, Develop, Validate, To Fix, To Release, Done, Blocked). Lo *stato* dice in che fase è una card.

L'assegnatario è il testimone. Ortogonale allo stato, l'*assignee* dice *di chi è il turno adesso*:

- assegnata all'**umano** — l'automazione la ignora (gate umano);
- assegnata al **bot** — l'automazione agisce;
- tenuta all'umano — intoccabile, per portarla avanti a mano.

Ogni passaggio è una riassegnazione. Così lo human-in-the-loop è **esplicito, tracciabile e visibile**: guardi la board e vedi subito cosa aspetta te e cosa è in mano agli agenti.

3. La pipeline

- **Intake** — una issue GitHub diventa una card *Bug*, assegnata all'umano per la triage.
- **Ideazione** — un agente legge il codice e propone idee ancorate a file reali, come card *Idea* da triagiare.
- **Planning** — per le card che l'umano affida al bot, un agente scrive un piano tecnico preciso come commento e le riconsegna. Con feedback loop: l'umano commenta, riassegna, e il piano viene *revisionato* recependolo.

- **Sviluppo** — un agente crea un branch `features/<id-card>` da `develop`, implementa il piano, esegue build/test come gate di qualità, pusha il branch e porta la card in *Validate* per la review umana (o *Blocked* con motivo preciso).
- **Rilascio su develop** — uno step deterministico apre e merge la PR della feature in `develop`; la card va in *Done*.
- **Rilascio in produzione** — uno step on-demand apre la PR `develop -> main`; l'umano la merge (è il code-owner) e la versione viene taggata. La produzione non la merge mai il bot da solo.

4. Perché funziona

- **Tiering dei modelli e gate anti-costo.** Ogni step schedulato esegue prima un *pre-check senza LLM*: se non c'è lavoro assegnato al bot, il modello non parte — paghi solo il lavoro vero. Il modello è configurabile per fase: economico ma capace per il coding, premium per il planning, gratuito per i task leggeri. L'architettura è agnostica sul modello: più il modello è intelligente, migliore è l'output — senza cambiare la pipeline.
- **Sicurezza per incapsulamento.** Gli agenti non toccano mai i segreti grezzi. Le credenziali stanno in un file `env` / nei secret CI e sono esposte solo tramite helper sottili e un git credential helper, così un modello “chiacchierone” non può mai stampare un token.
- **Trunk sicuro (GitFlow).** `develop` è integrazione (solo PR); `main` è produzione, protetto da una review code-owner che è l'umano. I contributor esterni forkano e aprono PR verso `develop` — su `main` non pusha nessuno tranne l'owner.
- **Self-hosted e portabile.** L'orchestratore gira sulla macchina dello sviluppatore (o su un server), parla ai modelli via un router di provider e a Jira/GitHub via API. Nessun lock-in; l'intera configurazione è portabile.

5. Benefici

- **Throughput scollegato dall'umano.** L'umano rivede decisioni, non battiture.
- **Controllo che scala.** Lo human-in-the-loop è imposto by design, visibile sulla board, sicuro di default.
- **Costo proporzionale al valore.** I cicli a vuoto costano zero.
- **A prova di futuro.** Man mano che i modelli migliorano, la qualità sale da sola: cambi un ID modello, non il processo.
- **Tracciabile.** Ogni azione è attribuita (bot vs umano), ogni piano e decisione vive sulla card.

6. Limiti onesti

- La qualità del codice segue la capacità del modello — un modello debole si avvia, uno capace consegna.
- Alcuni limiti di piattaforma sono reali (es. le restrizioni di push per-branch differiscono tra repo personali e di organizzazione) e vengono aggirati, non ignorati.
- I merge automatici sul branch di integrazione presuppongono che l'umano abbia già rivisto il codice; i merge in produzione restano manuali per scelta.
- Questa è una metodologia ingegneristica, non un risultato accademico: i claim sono architetturali, non uno studio con benchmark. Prossimo passo naturale: misurare throughput, costo per feature, tempo di review.

7. Conclusione

Agentic Scrum è una risposta pratica a “lascia scrivere il codice al modello, tieni il controllo all’umano”. Trasformando una board Jira nell’interfaccia umano-agente e l’*assegnatario* nel segnale di controllo, lo sviluppatore diventa l’orchestratore di un team AI — decide *cosa* e *se*, mentre gli agenti gestiscono il *come*. È disponibile oggi, economico da far girare, sicuro per costruzione, e migliora da solo man mano che i modelli diventano più intelligenti.

Licenza e citazione

Autore: Antonio Riccio — DomoticLab

Contatto: riccio.antonio19@gmail.com

Repository: <https://github.com/AntonioR199/Spoolio-opensource>

Versione: Whitepaper v1 — 8 luglio 2026

Licenza. Questo documento è distribuito con licenza Creative Commons Attribuzione 4.0 Internazionale (CC BY 4.0). Sei libero di condividerlo e adattarlo, anche per fini commerciali, a condizione di attribuire correttamente la paternità e di indicare eventuali modifiche. Testo completo della licenza: <https://creativecommons.org/licenses/by/4.0/deed.it>

Come citare. Riccio, A. (2026). *Agentic Scrum: lo sviluppatore come Product Owner, gli agenti AI come team*. DomoticLab. <https://github.com/AntonioR199/Spoolio-opensource>